LECTURES

03 Anonymous and arrow functions | 04 Closures | 10 Arrays | ★ Cheat sheet

REVISION NOTES · JAVASCRIPT

JavaScript Functions and Arrays Notes

Condensed from three decks: Anonymous function, Closures and Array. Code is cleaned up from the slides (curly quotes and small typos fixed). Points marked "Extra" go slightly beyond the slides.

Anonymous, callback, self-invoking and arrow functions

Deck 03 · Anonymous_function

Functions are values

A function definition can be assigned to a variable. The variable then points to the function, much like a function pointer in C or .NET.

```
var go = function(){ return "This is a String"; };   // anonymous function, go is its
var testStr = go();         // call with ()  -> "This is a String"
document.write(go);         // no ()  -> prints the function definition

var testCopy = go;          // copies the pointer, no () after go
var testing = testCopy();   // "This is a String"
```

- To call it, put `()` after the variable.
- Writing `testCopy = go` without brackets copies the **function**; `testCopy = go()` would store the **result**.

Passing a function to a function

```
let s = function(a){ return a*a; };
let c = function(a){ return a*a*a; };
var go = function(w, d){ return w(d); };   // my job: call whichever function you give

go(c, 2);    // 8
go(s, 2);    // 4
```

When `c` is passed, its value (the pointer) is copied into `w`. After the call, `c` and `w` both point to the same function definition.

Callback

Passing a function **definition** straight into the call is a callback. It works, but you lose the flexibility of reusing a named function.

```
var go = function(w){ alert(w(5)); };
go(function(a){ return a*a*a; });          // alerts 125
```

Check question from the deck:

```
function mydata(param1, param2, callback){
  alert('Started learning at.\n\nIt has: ' + param1 + ', ' + param2);
  callback();
}
mydata('vita', 'DAC', function(){ alert('Finished learning DAC'); });
```

OUTPUT

First alert: "Started learning at." then a blank line, then "It has: vita, DAC". Second alert: "Finished learning DAC". The callback runs last because `mydata` calls it after its own alert.

Self-invoking function (IIFE)

An anonymous function with no handle, followed by `()`, which calls it. It runs as soon as the page loads.

```
(function(){ alert("This is a String"); }())

var r = (function(a){ return a*a; }(5));    // 25, captured in r
document.write(r);
```

NAMED HANDLE	SELF-INVOKING
`var go = function(a){...}` `go(5)`	`(function(a){...}(5))`
Write `go` to see the definition; `go(5)` to call it	The `(5)` after the function is the call. `r` holds the **return value**, not the function

Why learn it: use it to initialise data as soon as the application starts. The jQuery library is written this way: `(function(window, undefined){ ... })(window)`.

Arrow functions

ANONYMOUS FUNCTION	ARROW FUNCTION
`function(value){ return value; }`	`value => value`
`function(num1, num2){ return num1 + num2; }`	`(num1, num2) => num1 + num2`
`function(){ return "TSYS"; }`	`() => "TSYS"`
`function(id){ return { id: id, name: "Temp" }; }`	`id => ({ id: id, name: "Temp" })`

- One parameter: brackets optional. No parameters or two or more: brackets required.
- A single expression after `=>` is returned automatically. With `{ }` you must write `return` yourself.
- To return an **object literal**, wrap it in round brackets, otherwise the braces are read as a function body.

```
let isEven = n => n % 2 == 0;             // returns true or false
isEven(4);                                // true

let isEven2 = n => { if (n % 2 == 0) return true; else return false; };

let factorial = (n) => { let f = 1; for (let i = 2; i <= n; i++) f = f * i; return f;
factorial(5);                             // 120

let cube = n => n*n*n;                    // cube(2) is 8

var sumfunction = (...arg) => {           // rest parameter works in arrows too
  let sum = 0;
  for (let i = 0; i < arg.length; i++) sum += arg[i];
  return sum;
};
sumfunction(5, 6, 3, 2, 1);               // 17
```

Extra: unlike normal functions, arrow functions have no `this` or `arguments` of their own.

Arrow functions with array methods

```
var values = [6, 2, 4, 5, 1, 9];
values.sort(function(a, b){ return a - b; });
values.sort((a, b) => a - b);            // same thing, shorter

[1, 4, 9, 16].map(x => x * 2);           // [2, 8, 18, 32]
```

`map()` builds a new array from the result of calling the function on every element. More on

`map()` builds a new array from the result of calling the function on every element. More on array methods below.

Closures

Deck 04 · closures

The problem: a counter

APPROACH	OUTPUT	PROBLEM
Global `counter`, function adds 1	1 2 3	Works, but any code can change the global
`let counter = 0` **inside** the function	1 1 1	Private now, but it resets to 0 on every call

```
let counter = 0;
function add(){ counter += 1; return counter; }
add(); add(); add();       // 1 2 3, but counter is global and can be modified

function add2(){ let counter = 0; counter += 1; return counter; }
add2(); add2(); add2();    // 1 1 1, counter is re-initialised each call
```

Nested functions

JavaScript supports a function inside a function. The inner function can use the variables of the outer function ("the scope above it").

```
function add(){
  var counter = 0;
  function plus(){ counter += 1; }
  plus();
  return counter;
}
add(); add();     // 1 1   still resets, and plus() cannot be reached from outside
```

Two things are still needed: reach the inner function from outside, and run `counter = 0` **only once**. That is a closure.

The closure

```
var add = (function(){
  var counter = 0;
  return function(){ return counter += 1; };
```

```
})();

add();   // 1
add();   // 2
add();   // 3
```

1. The outer function is self-invoking, so it runs once, when the page loads. `counter` is set to 0 only that one time.
2. It returns the inner anonymous function. `add` now points to that inner function (not to the result of the outer one).
3. Each call to `add()` runs the inner function, which still has access to `counter`, and increments it.
4. `counter` can be reached only through `add`, so nothing else can change it. It is effectively private.

DEFINITION

A closure is a function that keeps access to the variables of the scope where it was created, even after the outer function has finished running.

Used with a button

```
<button type="button" onclick="myFunction()">Count!</button>
<p id="demo">0</p>
<script>
  var add = (function(){ var counter = 0; return function(){ return counter += 1; }; }
  function myFunction(){ document.getElementById("demo").innerHTML = add(); }
</script>
```

Order in the deck: (1) the self-invoking function runs at load and sets up `counter`, (3) the user clicks, (4) `myFunction` runs, (5) it calls `add()` and shows the new count. The count goes up by 1 per click because `counter` survives between clicks.

WATCH OUT

The brackets `()` at the end of the outer function matter. Without them, `add` would hold the outer function itself, and calling it would reset the counter every time.

Arrays

Deck 10 · Array

Creating arrays

- An array is a collection of values of any type, because everything derives from `Object`. Internally it is stored as key and value pairs (index and value), and the array object lives on the heap.
- Declare with **square brackets**. `new Array()` works, but the slides say not to use it.

```
let a = [1, "ket", 'g', 20.5];       // preferred
let arr = new Array();               // avoid
arr[0] = 1; arr[1] = "vita"; arr[2] = 'g'; arr[3] = 20.5;
arr.length;                          // 4
document.write(arr);                 // 1,vita,g,20.5   (toString is overridden to joi
za.constructor;                      // function Array() { [native code] }
```

BE CAREFUL WITH `NEW ARRAY(N)`

Several numbers give the elements;
ONE
number gives the size.

```
new Array(3, 8, 9, 2)    // [3, 8, 9, 2]
var aa = new Array(2);   // size 2, empty
aa.length;               // 2
aa[0] = 7; aa[1] = 3;    // 7,3
```

Looping

LOOP	GIVES YOU	ADVICE
`for (let i = 0; i < arr.length; i++)`	Index	Classic, always fine
`for (let x of arr)`	Each **value**	Newer and preferred
`for (let x in arr)`	Each **index/key** that exists	The slides say do not use it for arrays

Gaps in an array

```
let arr = [];
arr[0] = 5;
arr[4] = 4;                          // length is now 5
for (let e of arr) document.write(e);          // 5 undefined undefined undefined 4
for (let e in arr) document.write(arr[e]);     // only keys that exist: 5 and 4
for (let e of arr) if (e) document.write(e);   // 5 4   (skip undefined)
```

```
for (let i = 0; i < arr.length; i++) document.write(i); // 0 4 (skip undefined)
```

Indexes 1, 2 and 3 have no key, so reading them gives `undefined`. Printing the whole array with `document.write(arr)` shows `5,,,,4`.

2D array

```
var a = new Array();
a[0] = new Array();  a[1] = new Array();
a[0][0] = "raj";   a[0][1] = 50;  a[0][2] = "A";
a[1][0] = "mona";  a[1][1] = 80;  a[1][2] = "B";

for (i = 0; i < a.length; i++){
  for (j = 0; j < a[i].length; j++) document.write(" " + a[i][j]);
  document.write("<hr/>");
}
// the same data as a literal: [["raj", 50, "A"], ["mona", 80, "B"]]
```

A 2D array is an array whose elements are arrays. `a[0][1]` is row 0, column 1, which is 50.

Method reference

Changes the original array:

METHOD	WHAT IT DOES	RETURNS
`push(x, ...)`	Adds to the **end**	New length
`pop()`	Removes the **last** element	The removed element
`unshift(x, ...)`	Adds to the **start**	New length (older IE did not return it)
`shift()`	Removes the **first** element	The removed element
`splice(index, howmany, items...)`	Removes and/or inserts at a position	Array of removed items (Extra)
`reverse()`	Reverses the order	The same array
`sort(compareFn)`	Sorts in place	The same array
`fill(value, start, end)`	Overwrites a range with one value	The same array

Leaves the original alone:

Leaves the original alone:

METHOD	WHAT IT DOES	RETURNS
`concat(other)`	Joins arrays	New array
`slice(begin, end)`	Copies a part	New array
`join(separator)`	Array to text	String
`map(fn)`	Transforms every element	New array, same length
`filter(fn)`	Keeps elements passing the test	New array
`find(fn)`	First element passing the test	The element, or `undefined`
`at(i)`	Element at an index, negatives count from the end	The element
`includes(x)`	Is the value present?	`true` / `false`

concat, slice, join

```
let nar = [10, 20, 30], ar = [5, 6, 7];
let result = ar.concat(nar);   // [5, 6, 7, 10, 20, 30]  ar and nar unchanged

var cut = [100, 200, 300, 400];
cut.slice(1);       // [200, 300, 400]   from index 1 to the end
cut.slice(1, 3);    // [200, 300]        end index is NOT included
cut;                // [100, 200, 300, 400]  unchanged

var ar2 = new Array(5, 3, 2);
ar2.join("-");      // "5-3-2"
ar2.join();         // "5,3,2"   default separator is a comma
typeof ar2.join();  // "string"
```

pop, push, shift, unshift

```
var arr = ["D", "A", "C"];
arr.pop();           // "C"       arr is ["D", "A"]

var arr2 = ["vita", "dac", "acts"];
arr2.push("2020");    // 4         arr2 is ["vita","dac","acts","2020"]

var nar = [100, 200, 300];
nar.shift();          // 100       nar is [200, 300], length 2
```

```
var arr3 = ["vita", "dac", "acts"];
arr3.unshift("2020"); // 4         arr3 is ["2020","vita","dac","acts"]
```

MEMORY AID

`push` / `pop` work at the **END**. `unshift` / `shift` work at the **START**. Adders (`push`, `unshift`) return the new length; removers (`pop`, `shift`) return the removed element.

reverse

```
var srt = new Array("anita", "zeena", "beena");
srt.reverse();            // beena, zeena, anita   the ORIGINAL is reversed
var scpy = srt.reverse(); // reversed again; scpy and srt are the same array
```

The slide ties this to shallow copy: like `let ob2 = ob` for objects, `scpy` is not a copy but a second name for the same array. A number is different: `a = 5; b = a;` gives `b` its own copy of the value.

splice(index, howmany, item1, ..., itemX)

PARAMETER	MEANING
`index`	Required. Position to add or remove at. Negative counts from the end
`howmany`	Optional. How many to remove. 0 removes nothing
`item1...`	Optional. Items to add

```
var fruits = ["Banana", "Orange", "Apple", "Mango"];
fruits.splice(1, 2);                // removes 2 from index 1 -> ["Banana", "Mango"]

var fr = ["Banana", "Orange", "Apple", "Mango"];
fr.splice(2, 0, "Lemon", "Kiwi");    // inserts at index 2
// ["Banana","Orange","Lemon","Kiwi","Apple","Mango"]
```

sort

- Sorts **in place**. The original array is changed; you need not assign the result.

- With no function, elements are converted to **strings** and sorted alphabetically (by character code, so uppercase comes before lowercase).

```
new Array("anita", "Zeena", "beena").sort();   // Zeena, anita, beena
[100, 350, 1, 2].sort();                       // 1, 100, 2, 350   (text order!)
[100, 350, 1, 2].sort((a, b) => a - b);        // 1, 2, 100, 350   numeric
[3, 2, 6, 50, 10].sort((a, b) => b - a);       // 50, 10, 6, 3, 2  descending
```

HOW THE COMPARE FUNCTION WORKS

The function gets two elements, `a` and `b`. Return a

NEGATIVE

number to put `a` first,

ZERO

to treat them as equal, a

POSITIVE

number to put `b` first (swap). So `a - b` sorts ascending and `b - a` sorts descending.

Engines differ: V8 uses quicksort or insertion sort for small arrays, Firefox uses merge sort, Safari varies by array type.

Sorting an array of objects: build a comparison key and return -1, 1 or 0.

```
let users = [
  {name: 'Scotty', age: '18'}, {name: 'Tommy', age: '21'}, {name: 'Sally', age: '71'},
  {name: 'Billy',  age: '18'}, {name: 'Timmy', age: '21'}
];
users.sort((a, b) => {
  let keyA = a.age + a.name;     // "18Scotty"
  let keyB = b.age + b.name;     // "21Tommy"
  if (keyA < keyB) return -1;
  if (keyA > keyB) return 1;
  return 0;
});
// Billy 18, Scotty 18, Timmy 21, Tommy 21, Sally 71
// Reverse the order by swapping the comparison, or by using keyB < keyA
```

Age sorts correctly here only because the ages are strings of equal length. Then ties are broken by name.

find, filter, at, includes, fill, map

```
// find: first match, or undefined
[5, 12, 8, 130, 44].find(el => el > 10);                    // 12
inventory.find(fruit => fruit.name === 'cherries');         // { name: 'cherries',
```

```
// filter: all matches, as a new array
['spray','limit','elite','exuberant','destruction','present'].filter(w => w.length > 6
// ["exuberant", "destruction", "present"]
[12, 5, 8, 130, 44].filter(v => v >= 10);                    // [12, 130, 44]

// at: negative index counts from the end
const a1 = [5, 12, 8, 130, 44];
a1.at(2);      // 8
a1.at(-2);     // 130

// includes: true or false, often used to check a user (authentication)
a1.includes(8);   // true

// fill(value, start, end): changes the original; end is not included
const f = ["Banana", "Orange", "Apple", "Mango"];
f.fill("Kiwi", 2, 4);    // Banana, Orange, Kiwi, Kiwi
f.fill("Kiwi", 1);       // from index 1 to the end: Banana, Kiwi, Kiwi, Kiwi (on a fr

// map: new array, original unchanged, skips empty slots
[1, 4, 9, 16].map(x => x * 2);              // [2, 8, 18, 32]
[1, 4, 9, 16].map(x => "<b>" + x + "</b>");  // wrap each in bold tags
```

MAP	FILTER	FIND
Same length, each element transformed	Shorter or equal, elements unchanged	One element (or `undefined`)

Last-minute cheat sheet

All three decks

Likely exam questions

- What is an anonymous function; how to call it; difference between `go` and `go()`.
- What is a callback; write the output of the `mydata` example.
- What is a self-invoking function and why it is used (jQuery, initialisation).
- Convert an anonymous function to an arrow function; when braces and `return` are needed.
- Write a closure counter and explain why it prints 1, 2, 3 while the plain local version prints 1, 1, 1.
- Ways to create an array; `new Array(3)` vs `new Array(3, 8, 9)`.
- `for of` vs `for in`; what happens with gaps.

- Difference between `slice` and `splice`; between `shift` and `pop`.
- Why `[100, 350, 1, 2].sort()` gives 1, 100, 2, 350, and how to fix it.
- Which methods change the original array and which return a new one.

Easy mistakes

- `var r = go` (copies the function) vs `var r = go()` (stores the result).
- Forgetting the final `()` in the closure, which breaks the counter.
- Returning an object literal from an arrow function without wrapping it in `( )`.
- Using `sort()` on numbers without a compare function.
- Treating `slice` as if it changes the array, or `splice` as if it does not. Slice copies, splice edits.
- Assuming `slice(1, 3)` includes index 3. The end index is excluded, and so is the end of `fill`.
- Thinking `b = a` copies an array. It gives a second name for the same array (also after `reverse`).
- Using `for in` over an array and getting keys, not values.